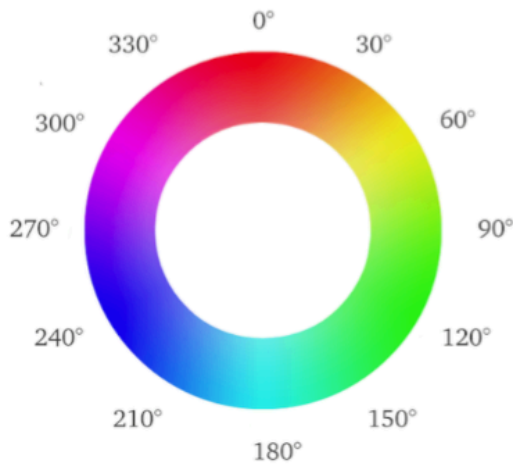
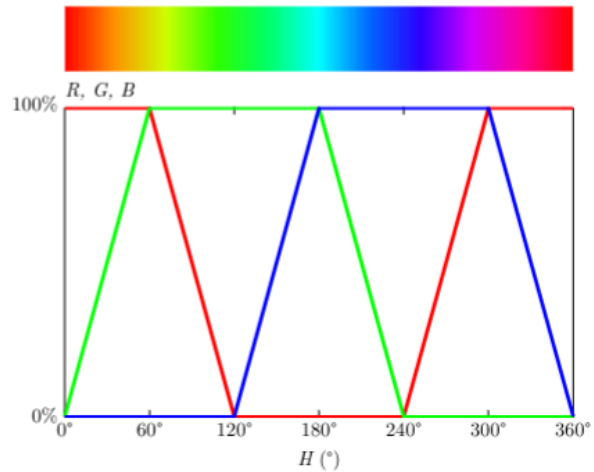


ENGR 3410: Miniproject 2

I built a digital circuit using the OSS CAD suite and iceBlinkPico board to cycle through the colors of the HSV color wheel once per second using pulse width modulation (PWM) according to the following waveforms:



The HSV color wheel.



RGB components versus hue angle, H .

The board itself was created with Lattice Semiconductor's iCE40UP5K FPGA utilizing the Nitro FPGA bootloader. After encoding my digital circuit, I uploaded it onto the board by navigating into the project directory, entering the bootloader mode, and building my files using the "make" and "make prog" commands. All programming was done in Linux using System Verilog.

Color States

Since the circuit must loop through the HSV color wheel in a smooth and continuous loop, I divided this 360 degree cycle into six 60 degree segments (or states). Under each state, one color increases, one decreases, and the third is held constant (at ON/maximum PWM value or OFF/minimum PWM value).

*The RGB LED is configured as active-low; therefore, it is off when the driving FPGA pin is high and on when it is low.

Hue Range	R	G	B	Color Transition
0° - 60°	Max	Increase	Min	Red to yellow
60° - 120°	Decrease	Max	Min	Yellow to green

120° - 180°	Min	Max	Increase	Green to cyan
180°-240°	Min	Decrease	Max	Cyan to blue
240° - 300°	Increase	Min	Max	Blue to magenta
300° - 360°	Max	Min	Decrease	Magenta to red

Timing Parameters

A 12-MHz crystal oscillator acts as the clock input for the iceBlinkPico. The timing constraint for a full color cycle was 1 second.

```
// Default clock frequency
CLK_FREQ = 12,000,000
// Maximum PWM resolution
// Ensures LED at continuous brightness (not flickering)
PWM_MAX = 1200
// States transition every ~0.17s
// 6 states per clock cycle
STATE_DURATION = 2,000,000
// PWM values for RGB incremented/decremented every ~0.00014s
// 1200 steps per state
UPDATE_INTERVAL = 1666.666
```

Source Code:

<https://github.com/eneri4/iceBlinkPico/tree/main/examples>

All of my source code is in the github repository above under the RGB_LED folder. The digital circuit is partitioned into three modules: top, RGB_LED, and pwm.

The pwm module converts the duty cycle values (which corresponds to the desired brightness level) into a square-wave output signal (creating PWM). An up-counter (pwm_count) is used to cycle from 0 to 1199 (our PWM resolution). Given the 12 MHz clock and a PWM_INTERVAL of 1200, the PWM frequency is 10 kHz, which is sufficiently fast to prevent visible flicker. PWM assignments are determined by the following logic:

```
pwm_out=(pwm_count<pwm_value)?1'b0:1'b1
```

If the counter is less than the desired pwm_value, which is set by the controller RGB_LED, then pwm_out is asserted low (or ON). If the counter is greater than pwm_value, then pwm_out is

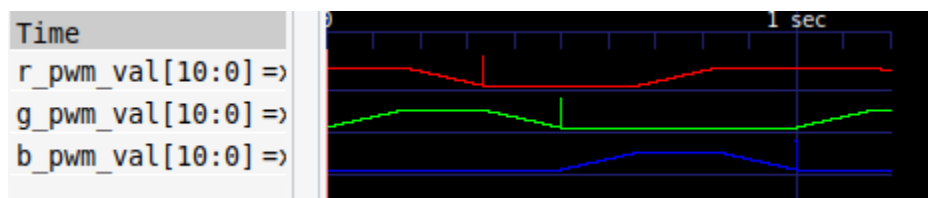
asserted high (or OFF). This ensures that the active-low logic of the RGB_LED is followed since the output pin will be driven low for nearly the entire 1200 cycle period if pwm_value commands maximum brightness.

The RGB_LED module manages the timing and brightness transitions for the entire color cycle. It uses a Finite State Machine with the color states defined above. The module's parameters dictate the 1-second cycle time. The total 12,000,000 clock cycles in one second are divided evenly among the six states, setting the STATE_DURATION to exactly 2,000,000 cycles. The UPDATE_INTERVAL determines how frequently the duty cycle values (r_val, g_val, b_val) are changed by a single step. This interval is calculated as STATE_DURATION/PWM_MAX, which is approximately 1666.67 cycles. A continuous counter generates an update_tick pulse based on the UPDATE_INTERVAL. Upon this tick, the FSM directs one color value to linearly increment and one to linearly decrement between 0 and 1199. This control guarantees a smooth ramp over the 2,000,000 cycles of each state.

The top module is the structural layer that connects the lower-level components to the external FPGA pins. It instantiates the RGB_LED controller and three separate PWM modules (one for each color channel), routing the calculated duty cycle values (r_pwm_val, g_pwm_val, b_pwm_val) to the pwm outputs (r_pwm_out, g_pwm_out, and b_pwm_out), which in turn drive the final RGB_R, RGB_G, and RGB_B output pins.

Simulation

The SystemVerilog test bench RGB_LED_tb.sv was configured to simulate the environment accurately. It generates a perfect 12 MHz clock signal and runs the simulation for 1.2 seconds. This duration is sufficient to observe one complete 1-second color cycle and the beginning of the next cycle, verifying the state transitions and the linear ramping of the brightness values.



The waveforms as shown in the chart from GTKWave above (using the step analog data format) confirm the intended phase shifts between RGB values that cause the HSV transitions.